НЕГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ ЧАСТНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«МОСКОВСКИЙ ФИНАНСОВО-ПРОМЫШЛЕННЫЙ УНИВЕРСИТЕТ «СИНЕРГИЯ»

Кафедра Инновационных и цифровых технологий

ЛАБОРАТОРНАЯ РАБОТА № 4

Обучающийся Волков Илья Викторович

(ФИО) (подпись)

Преподаватель Сибирев Иван Валерьевич

(ФИО) (подпись)

Москва 2026 г.

СОДЕРЖАНИЕ

Введение 3

1. Системы контроля версий и Git 3

2. Базовые операции Git 4

3. GitHub: облачный хостинг для разработчиков 5

4. GitLab: единая DevOps-платформа 6

5. Работа с GitLab CI/CD 7

6. Gitea: лёгкая self-hosted альтернатива 8

7. Сравнительный анализ GitHub, GitLab и Gitea 9

Заключение 9

Список использованных источников 10

ВВЕДЕНИЕ

Разработка современного программного обеспечения практически немыслима без применения систем контроля версий. Эти инструменты позволяют отслеживать историю изменений кода, организовывать параллельную работу множества разработчиков, откатывать ошибочные изменения и поддерживать различные варианты продукта одновременно. Среди всего многообразия систем контроля версий безусловным лидером является Git, а ведущими платформами, построенными на его основе, — GitHub, GitLab и набирающая популярность альтернатива Gitea.

Эти платформы давно вышли за пределы простого хранения репозиториев кода. Сегодня они представляют собой полноценные экосистемы, объединяющие в себе функции управления проектами, код-ревью, непрерывной интеграции, автоматического развёртывания и мониторинга. Выбор конкретной платформы существенно влияет на организацию всего DevOps-процесса в компании, поэтому понимание их особенностей и отличий необходимо каждому современному разработчику.

Цель данной работы — рассмотреть систему контроля версий Git и три наиболее распространённые платформы, построенные на её основе: GitHub, GitLab и Gitea. В работе будут изучены базовые принципы работы Git, особенности каждой из платформ, их сильные и слабые стороны, а также проведён сравнительный анализ для понимания области применения каждого решения.

1. СИСТЕМЫ КОНТРОЛЯ ВЕРСИЙ И GIT

Система контроля версий (Version Control System, VCS) — это программное обеспечение, предназначенное для отслеживания изменений в файлах с течением времени и организации совместной работы над ними. VCS позволяет в любой момент восстановить любую из сохранённых версий файла, увидеть, кто, когда и какие именно изменения внёс, а также объединять изменения, сделанные разными участниками параллельно.

Исторически системы контроля версий прошли несколько этапов развития. Первое поколение — локальные VCS (SCCS, RCS) — работали на одной машине. Второе поколение — централизованные системы (CVS, SVN, Perforce) — хранили историю на едином сервере, к которому обращались клиенты. Третье, современное поколение — распределённые VCS (Git, Mercurial), в которых каждый разработчик имеет полную копию репозитория с всей историей изменений.

Git — распределённая система контроля версий, созданная Линусом Торвальдсом в 2005 году для разработки ядра Linux. Git был разработан с учётом требований к скорости, надёжности, поддержке нелинейной разработки (тысячи параллельных веток) и распределённого характера работы. Благодаря сочетанию высокой производительности, мощной модели ветвления и открытого исходного кода Git быстро вытеснил своих конкурентов и сегодня является де-факто стандартом в индустрии.

Ключевые особенности Git:

- Распределённая архитектура — каждый клон репозитория содержит полную историю изменений;
- Целостность данных — каждая версия идентифицируется криптографическим хешем SHA-1;
- Лёгкое ветвление и слияние — создание новой ветки занимает миллисекунды;
- Работа преимущественно офлайн — большинство операций выполняется локально;
- Стейджинг (область индекса) — промежуточный этап перед фиксацией изменений.

Важно понимать отличие Git от перечисленных далее платформ. Сам по себе Git — это инструмент командной строки, работающий с локальными репозиториями. Для организации совместной работы нескольких

разработчиков требуется удалённый сервер, к которому все участники имеют доступ. Именно эту роль и выполняют платформы GitHub, GitLab и Gitea.

2. БАЗОВЫЕ ОПЕРАЦИИ GIT

Работа с Git основана на небольшом наборе базовых операций, которые в совокупности покрывают подавляющее большинство задач повседневной разработки.

Инициализация и клонирование. Новый репозиторий создаётся командой git init, которая создаёт скрытую директорию .git с метаданными. Существующий удалённый репозиторий копируется на локальную машину командой git clone.

Фиксация изменений. Процесс состоит из двух этапов: сначала изменённые файлы добавляются в область индекса командой git add, затем фиксируются в истории командой git commit с пояснительным сообщением. Такая двухэтапная модель позволяет тщательно контролировать, что именно попадает в коммит.

Ветвление. Ветка в Git — это легковесный указатель на конкретный коммит. Создание новой ветки (git branch) и переключение на неё (git checkout или git switch) выполняются практически мгновенно. Модель ветвления Git настолько эффективна, что поощряет создание веток для каждой новой задачи или эксперимента.

Слияние и ребейз. Объединение изменений из разных веток осуществляется командой git merge (создаёт merge-коммит) или git rebase (переписывает историю, делая её линейной). Выбор между этими подходами определяется принятой в команде стратегией.

Синхронизация с удалённым сервером. Команды git fetch и git pull используются для получения обновлений из удалённого репозитория, а git push — для отправки локальных изменений обратно.

Типичная последовательность работы над новой функциональностью выглядит следующим образом:

```
git clone https://gitlab.example.com/project.git
cd project
git checkout -b feature/new-login
# внесение изменений в код
git add .
git commit -m "Implement new login form"
git push origin feature/new-login
```

После отправки ветки на сервер разработчик создаёт запрос на слияние (pull request или merge request в зависимости от платформы), по которому коллеги проводят код-ревью, после чего изменения попадают в основную ветку.

3. GITHUB: ОБЛАЧНЫЙ ХОСТИНГ ДЛЯ РАЗРАБОТЧИКОВ

GitHub — самая известная в мире платформа для хостинга Git-репозиториев, основанная в 2008 году и приобретённая корпорацией Microsoft в 2018 году. GitHub первым популяризировал концепцию социального программирования, превратив разработку открытого программного обеспечения в публичный процесс с возможностью обсуждений, звёздочек и форков.

Ключевыми возможностями GitHub являются:

- Хостинг публичных и приватных репозиториев с неограниченным количеством участников;
- Система Pull Request — основной инструмент код-ревью и обсуждения изменений;
- Issue Tracker — система управления задачами и ошибками;
- GitHub Actions — встроенная система CI/CD с поддержкой workflow-файлов;

- GitHub Marketplace — каталог из десятков тысяч переиспользуемых действий (actions);
- GitHub Packages — реестр пакетов для различных языков и Docker-образов;
- GitHub Pages — бесплатный хостинг статических сайтов непосредственно из репозитория;
- Copilot — интеграция с ИИ-ассистентом для написания кода.

GitHub Actions, встроенная CI/CD-система GitHub, позиционируется как отдельный инструмент, интегрированный в платформу. Конфигурация описывается в YAML-файлах, размещаемых в каталоге .github/workflows — каждый файл описывает отдельный workflow: тесты, деплой, линтеры. Такое разделение по файлам удобно для крупных проектов, но может усложнять обзор всей CI/CD-логики.

GitHub — полностью облачное решение, размещённое на серверах Microsoft. Для корпоративных клиентов, которым требуется собственная установка, существует отдельный продукт GitHub Enterprise Server, устанавливаемый на инфраструктуру заказчика. Благодаря огромному сообществу, удобному интерфейсу и тесной интеграции с современными инструментами разработки GitHub остаётся бесспорным лидером в сегменте open source и стартапов.

4. GITLAB: ЕДИНАЯ DEVOPS-ПЛАТФОРМА

GitLab — веб-платформа для управления Git-репозиториями и всем жизненным циклом разработки программного обеспечения, созданная в 2011 году Дмитрием Запорожцем и Валерием Сизовым. В отличие от GitHub, позиционирующего себя как социальная сеть для разработчиков с дополнительными инструментами, GitLab с самого начала разрабатывался как *единая DevOps-платформа*, объединяющая в одном продукте все необходимые для разработки инструменты.

В GitLab нет разделения на отдельные продукты для разных задач — это единое интегрированное решение, включающее:

- Хостинг Git-репозиториев с развитой системой прав доступа;
- Систему Merge Request для код-ревью;
- Issue Tracker и канбан-доски для управления задачами;
- GitLab CI/CD — глубоко встроенную систему непрерывной интеграции и доставки;
- Container Registry для хранения Docker-образов;
- Package Registry для различных типов пакетов;
- Встроенные инструменты безопасности: SAST, DAST, анализ зависимостей;
- Средства мониторинга и метрики производительности;
- GitLab Pages для хостинга статических сайтов.

GitLab распространяется в трёх вариантах: облачный сервис GitLab.com, бесплатная Community Edition (CE) с открытым исходным кодом и коммерческая Enterprise Edition (EE) с расширенным функционалом. Возможность self-hosted установки является одним из ключевых преимуществ GitLab — она позволяет организациям с повышенными требованиями к безопасности полностью контролировать как код, так и метаданные о процессах разработки.

Стоит отметить терминологические особенности GitLab. Если в GitHub запросы на слияние называются Pull Request (поскольку первое действие — получить ветку локально), то в GitLab используется термин Merge Request, отражающий финальное действие — слияние веток. Эта различие не просто стилистическое — оно отражает разные философские подходы к совместной работе.

5. РАБОТА С GITLAB CI/CD

GitLab CI/CD — одно из ключевых преимуществ платформы GitLab, делающее её особенно привлекательной для DevOps-команд. В отличие от GitHub, где CI/CD-система работает как отдельный интегрированный инструмент, GitLab CI является неотъемлемой и базовой частью платформы.

Вся конфигурация CI/CD описывается в единственном файле .gitlab-ci.yml, размещаемом в корне репозитория. Такой подход имеет практическое преимущество: вся CI/CD-логика проекта находится в одном месте, и общая картина пайплайна видна сразу. Для крупных проектов существует механизм include, позволяющий выносить части конфигурации в отдельные файлы.

Базовая терминология GitLab CI:

- Pipeline — полный процесс выполнения CI/CD, запускаемый событиями репозитория;
- Stages — последовательные этапы, выполняющиеся один за другим;
- Jobs — отдельные задания, выполняющиеся параллельно в рамках одного stage;
- Runner — исполнитель, запускающий задания в изолированных окружениях;
- Artifacts — файлы, сохраняемые после выполнения jobs и доступные для последующих этапов;
- Tags — метки runner-ов, позволяющие точно выбирать окружение выполнения заданий.

Концепция Runner-ов в GitLab более инфраструктурна по сравнению с GitHub Actions. Если в GitHub Actions обычно указывается runs-on: ubuntu-latest с выбором среды из готовых вариантов, то в GitLab runner-ы разворачиваются самостоятельно на собственных серверах, виртуальных машинах или в Docker-контейнерах, после чего им назначаются tags. В задании явно указываются нужные tags, и runner с соответствующими метками берёт

задачу в работу. Такой подход даёт больше контроля над выполнением, но требует более осознанной настройки.

Рассмотрим пример простого пайплайна, демонстрирующий основные возможности GitLab CI:

```
stages:
  - build
  - test
  - deploy

variables:
  APP_NAME: "my-app"

default:
  image: python:3.11
  tags:
    - docker

install_dependencies:
  stage: build
  script:
    - pip install -r requirements.txt
  artifacts:
    paths:
      - .venv/
    expire_in: 1 day

run_tests:
  stage: test
  script:
    - pytest tests/
  needs: [install_dependencies]
```

```
deploy_to_production:
  stage: deploy
  script:
    - ./scripts/deploy.sh
  only:
    - main
  when: manual
```

В примере используются ключевые возможности GitLab CI: этапы (stages), глобальные переменные (variables), значения по умолчанию (default), артефакты (artifacts) и явные зависимости между заданиями (needs). Параметр when: manual указывает, что развёртывание в продакшен требует ручного подтверждения, что является типичной практикой для защиты от случайных выкладок.

Для секретов и чувствительных данных GitLab предоставляет раздел Settings → CI/CD → Variables, где переменные можно пометить как protected (доступны только в защищённых ветках) или masked (значения скрываются в логах). Это стандартный и безопасный способ хранения токенов, ключей и паролей.

6. GITEA: ЛЁГКАЯ SELF-HOSTED АЛЬТЕРНАТИВА

Gitea — легковесная платформа для хостинга Git-репозиториев с открытым исходным кодом, являющаяся форком проекта Gogs и активно развивающаяся с 2016 года. Основная цель Gitea — предоставить простое и нетребовательное к ресурсам решение для размещения Git-репозиториев на собственной инфраструктуре. Платформа написана на языке Go, благодаря чему распространяется как единый исполняемый файл и способна работать даже на устройствах начального уровня, включая Raspberry Pi.

Ключевые особенности Gitea:

- Минимальные системные требования — менее 1 ГБ оперативной памяти;
- Простая установка — запуск одного бинарного файла без сложных зависимостей;
- Лицензия MIT — полностью свободное программное обеспечение без платных редакций;
- Поддержка всех основных СУБД (PostgreSQL, MySQL, SQLite, MSSQL);
- Веб-интерфейс, приближённый к GitHub по удобству использования;
- Встроенный Issue Tracker, Pull Request и Wiki;
- Gitea Actions — совместимый с GitHub Actions синтаксис CI/CD, появившийся в версии 1.19;
- Поддержка интеграции через webhooks и REST/API-интерфейс.

Появление Gitea Actions в 2023 году стало значимой вехой для проекта. Новая система CI/CD использует тот же синтаксис YAML-файлов, что и GitHub Actions, что существенно упрощает миграцию между платформами. Пользователи могут использовать многие существующие GitHub Actions с минимальными изменениями или без них, что делает Gitea особенно привлекательной для команд, ранее работавших с GitHub.

Типичные области применения Gitea:

- Небольшие команды и отдельные разработчики, нуждающиеся в простом self-hosted Git-сервере;
- Домашние лаборатории (homelab) и личные проекты;
- Образовательные учреждения с ограниченными вычислительными ресурсами;
- Ситуации, когда GitLab избыточен по функциональности и ресурсам;
- Среды с особыми требованиями к изоляции от интернета (военные, государственные).

Несмотря на свою простоту, Gitea имеет и ограничения. Она не обладает такой развитой функциональностью встроенной безопасности и DevOps-инструментов, как GitLab, и не имеет такого масштабного сообщества, как GitHub. Однако для многих сценариев использования её возможностей более чем достаточно, а простота сопровождения становится решающим аргументом.

7. СРАВНИТЕЛЬНЫЙ АНАЛИЗ GITHUB, GITLAB И GITEA

Для удобства восприятия сведём ключевые характеристики рассмотренных платформ в единую таблицу:

Критерий	GitHub	GitLab	Gitea
Год выпуска	2008	2011	2016
Модель	SaaS, Enterprise Server	SaaS, self-hosted (CE/EE)	Self-hosted, open source
Язык реализации	Ruby on Rails	Ruby on Rails, Go	Go
Встроенный CI/CD	GitHub Actions	GitLab CI/CD	Gitea Actions
Требования к ресурсам	N/A (облако)	4+ ГБ RAM	менее 1 ГБ RAM
Тип запроса слияния	Pull Request	Merge Request	Pull Request
Open Source ядро	Нет	Да (CE)	Да (MIT)
Типичная аудитория	Open source, стартапы	Корпоративный DevOps	Малые команды, хоумлаб

Каждая из рассмотренных платформ имеет свою нишу применения. GitHub оптимален для open source проектов и команд, ориентированных на

взаимодействие с глобальным сообществом разработчиков. Огромная база готовых GitHub Actions в Marketplace, интеграция с современными облачными сервисами и инструментами делают его лучшим выбором для проектов, которым важна интеграционная экосистема. GitLab оптимален для корпоративных команд, которым требуется единая DevOps-платформа со всеми инструментами «из коробки», а также возможность полного контроля над инфраструктурой через self-hosted развёртывание. Gitea — выбор для небольших команд, личных проектов и ситуаций с ограниченными ресурсами, где полная функциональность GitLab избыточна.

Следует отметить, что платформы не являются взаимоисключающими. В крупных организациях нередко встречается гибридное использование: основная разработка ведётся на GitLab, а зеркала критичных open source проектов размещаются на GitHub для привлечения внешних контрибьюторов. Отдельные небольшие проекты или внутренние инструменты могут использовать Gitea как более простое и лёгкое решение.

ЗАКЛЮЧЕНИЕ

В ходе выполнения настоящей работы были рассмотрены система контроля версий Git и три основные платформы, построенные на её основе: GitHub, GitLab и Gitea. Установлено, что Git как распределённая система контроля версий образует технологическую основу современной разработки программного обеспечения, а платформы на его основе расширяют эту основу развитыми инструментами совместной работы, код-ревью и автоматизации.

GitHub остаётся лидером в сегменте open source благодаря огромному сообществу и богатой экосистеме интеграций. GitLab позиционируется как единая DevOps-платформа со встроенными инструментами CI/CD, безопасности и управления развёртыванием, что делает его популярным выбором в корпоративной среде. Gitea, являясь легковесной open source альтернативой, занимает нишу простых self-hosted решений для малых команд

и ограниченной инфраструктуры, при этом сохраняя совместимость с современными CI/CD-практиками через Gitea Actions.

Особое внимание в работе уделено GitLab CI/CD как наиболее развитой встроенной системе непрерывной интеграции. Рассмотрены базовые концепции pipeline, stages, jobs и runner-ов, приведён практический пример конфигурационного файла .gitlab-ci.yml. Показано, что осознанный выбор между платформами должен основываться на конкретных требованиях проекта: масштабе команды, потребностях в DevOps-инструментах, требованиях к безопасности данных и доступных ресурсах.

Таким образом, Git и построенные на его основе платформы представляют собой фундаментальный инструмент современной разработки программного обеспечения, от правильного выбора которого во многом зависит эффективность работы команды и качество итогового продукта.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Чакон С., Штрауб Б. Pro Git. — 2-е изд. — Apress, 2014. — 458 с.
2. Ким Дж., Дебуа П., Уиллис Дж., Хамбл Дж. Руководство по DevOps. — М.: Манн, Иванов и Фербер, 2018. — 416 с.
3. GitLab: что это такое и как им пользоваться [Электронный ресурс] // Skillbox Media. — URL: https://skillbox.ru/media/code/gitlab-chto-eto-takoe-i-kak-im-polzovatsya/ (дата обращения: 24.04.2026).
4. GitLab: Основы написания Pipeline 1/3 [Электронный ресурс] // Хабр. — URL: https://habr.com/ru/articles/980478/ (дата обращения: 24.04.2026).
5. GitLab: Основы написания Pipeline 2/3 [Электронный ресурс] // Хабр. — URL: https://habr.com/ru/articles/981658/ (дата обращения: 24.04.2026).
6. Официальная документация Git [Электронный ресурс]. — URL: https://git-scm.com/doc (дата обращения: 24.04.2026).

7. Официальная документация GitHub [Электронный ресурс]. — URL: https://docs.github.com (дата обращения: 24.04.2026).
8. Официальная документация GitLab [Электронный ресурс]. — URL: https://docs.gitlab.com (дата обращения: 24.04.2026).
9. Официальная документация Gitea [Электронный ресурс]. — URL: https://docs.gitea.com (дата обращения: 24.04.2026).
10. Торвальдс Л., Хамано Дж. Git — Fast Version Control System [Электронный ресурс]. — URL: https://git-scm.com (дата обращения: 24.04.2026).